Отчёт по лабораторной работе №7

Дисциплина: Компьютерный практикум по статистическому анализу данных

Выполнил: Танрибергенов Эльдар (НПИбд-01-22)

Содержание

1 Цель работы **5**

2 Предварительные сведения **6**

3 Выполнение лабораторной работы **7**

3.1 Запись данных в файл 9
3.2 Словари 10
3.3 DataFrames 12
3.4 RDatasets 13
3.5 Работа с переменными отсутствующего типа (Missing Values) 14
3.6 FileIO 16
3.7 Обработка данных: стандартные алгоритмы машинного обучения в Julia 17

4 Выводы **21**

Список иллюстраций

Список таблиц

1 Цель работы

Основной целью работы является изучение специализированных пакетов Julia для обработки данных.

2 Предварительные сведения

Обработка и анализ данных, полученных в результате проведения исследований, — важная и неотъемлемая часть исследовательской деятельности. Большое значение имеет выявление определённых связей и закономерностей в имеющихся неструктурированных данных, особенно в данных больших размерностей. Выявленные в данных связей и закономерностей позволяет строить прогнозные модели с предполагаемым результатом. Для решения таких задач применяют методы из таких областей знаний как математическая статистика, программирование, искусственный интеллект, машинное обучение. В Julia для обработки данных используются наработки из других языков программирования, в частности, из R и Python. Перед тем, как начать проводить какие-либо операции над данными, необходимо их откуда-то считать и возможно сохранить в определённой структуре. Довольно часто данные для обработки содержаться в csv-файле, имеющим текстовый формат, в котором данные в строке разделены, например, запятыми, и соответствуют ячейкам таблицы, а строки данных соответствуют строкам таблицы. Также данные могут быть представлены в виде фреймов или множеств.

3 Выполнение лабораторной работы

В Julia для работы с такого рода структурами данных используют пакеты CSV, DataFrames, RDatasets, FileIO:

[1]:
```
# Обновление окружения:
using Pkg
Pkg.update
# Установка пакетов:
using Pkg
for p in ["CSV", "DataFrames", "RDatasets", "FileIO"]
Pkg.add(p)
end
using CSV, DataFrames, DelimitedFiles
```

```
   Resolving package versions...
   Installed InlineStrings ── v1.4.5
   Installed SentinelArrays ─ v1.4.9
   Installed PooledArrays ─── v1.4.3
   Installed WorkerUtilities - v1.6.1
   Installed WeakRefStrings ─ v1.4.2
   Installed FilePathsBase ── v0.9.24
   Installed CSV ──────────── v0.10.16
  Installing 1 artifacts
```

Предположим, что у вас в рабочем каталоге с проектом есть файл с данными programminglanguages.csv, содержащий перечень языков программирования и год их создания. Тогда для заполнения массива данными для последующей обработки требуется считать данные из исходного файла и записать их в соответствующую структуру:

```
[9]: # Считывание данных и их запись в структуру:
P = CSV.File("C:programminglanguages.csv") |> DataFrame
```

[9]: 73×2 DataFrame

Row	year	language
	Int64	String31
1	1951	Regional Assembly Language
2	1952	Autocode
3	1954	IPL
4	1955	FLOW-MATIC
5	1957	FORTRAN
6	1957	COMTRAN

Далее приведём пример функции, в которой на входе указывается название языка программирования, а на выходе — год его создания:

```
[4]: # Функция определения по названию языка программирования года его создания:
function language_created_year(P,language::String)
loc = findfirst(P[:,2].==language)
return P[loc,1]
end
```

В следующем примере при вызове функции, в качестве аргумента которой указано слово julia, написанное со строчной буквы выдана ошибка, так как список не содержит таких данных:

```
[5]: # Пример вызова функции и определение даты создания языка Python:
language_created_year(P,"Python")
```

```
[5]: 1991
```

```
[6]: # Пример вызова функции и определение даты создания языка Julia:
language_created_year(P,"Julia")
```

```
[6]: 2012
```

Для того, чтобы убрать в функции зависимость данных от регистра, необходимо изменить исходную функцию следующим образом:

```
[7]: language_created_year(P,"julia")
```

```
MethodError: no method matching getindex(::DataFrame, ::Nothing, ::Int64)
The function `getindex` exists, but no method is defined for this combination of argument types.

Closest candidates are:
  getindex(::DataFrame, ::typeof(!), ::Union{Signed, Unsigned})
   @ DataFrames C:\Users\eldar\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:548
  getindex(::DataFrame, ::Colon, ::Union{AbstractString, Signed, Symbol, Unsigned})
   @ DataFrames C:\Users\eldar\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:542
  getindex(::DataFrame, ::AbstractVector, ::Union{AbstractString, Signed, Symbol, Unsigned})
   @ DataFrames C:\Users\eldar\.julia\packages\DataFrames\b4w9K\src\dataframe\dataframe.jl:530
  ...


Stacktrace:
 [1] language_created_year(P::DataFrame, language::String)
   @ Main .\In[4]:4
 [2] top-level scope
   @ In[7]:1
 [3] eval(m::Module, e::Any)
   @ Core .\boot.jl:489
```

Можно считывать данные построчно, с элементами, разделенными заданным разделителем:

```
[8]: # Функция определения по названию языка программирования
     # года его создания (без учёта регистра):
     function language_created_year_v2(P,language::String)
     loc = findfirst(lowercase.(P[:,2]).==lowercase.(language))
     return P[loc,1]
     end
     # Пример вызова функции и определение даты создания языка julia:
     language_created_year_v2(P,"julia")
```

```
[8]: 2012
```

3.1 Запись данных в файл

Предположим, что требуется записать имеющиеся данные в файл. Для записи данных в формате CSV можно воспользоваться следующим вызовом:

Можно задать тип файла и разделитель данных:

```
[10]: # Построчное считывание данных с указанием разделителя:
      Tx = readdlm("programminglanguages.csv", ',')
```

```
[10]: 74×2 Matrix{Any}:
          "year"  "language"
       1951       "Regional Assembly Language"
       1952       "Autocode"
       1954       "IPL"
       1955       "FLOW-MATIC"
       1957       "FORTRAN"
       1957       "COMTRAN"
       1958       "LISP"
       1958       "ALGOL 58"
       1959       "FACT"
       1959       "COBOL"
       1959       "RPG"
       1962       "APL"
          ⋮
       2003       "Scala"
       2005       "F#"
       2006       "PowerShell"
       2007       "Clojure"
       2009       "Go"
       2010       "Rust"
       2011       "Dart"
       2011       "Kotlin"
       2011       "Red"
       2011       "Elixir"
       2012       "Julia"
       2014       "Swift"
```

Можно проверить, используя readdlm, корректность считывания созданного текстового файла:

```
[11]: CSV.write("programming_languages_data2.csv", P)
```

```
[11]: "programming_languages_data2.csv"
```

3.2 Словари

При работе с данными бывает удобно записать их в формате словаря. Предположим, что словарь должен содержать перечень всех языков программирования и года их создания, при этом при указании года выводить все языки програм-

мирования, созданные в этом году. При инициализации словаря можно задать конкретные типы данных для ключей и значений:

```
[13]: # Пример записи данных в текстовый файл с разделителем ',':
      writedlm("programming_languages_data.txt", Tx, ',')
      # Пример записи данных в текстовый файл с разделителем '-':
      writedlm("programming_languages_data2.txt", Tx, '-')
```

В последнем случае словарь принимает ключи и значения любого типа. Далее требуется заполнить словарь ключами и годами, которые содержат все языки программирования, созданные в каждом году, в качестве значений:

```
[14]: # Построчное считывание данных с указанием разделителя:
      P_new_delim = readdlm("programming_languages_data2.txt", '-')
```

```
[14]: 74×2 Matrix{Any}:
          "year"  "language"
       1951       "Regional Assembly Language"
       1952       "Autocode"
       1954       "IPL"
       1955       "FLOW-MATIC"
       1957       "FORTRAN"
       1957       "COMTRAN"
       1958       "LISP"
       1958       "ALGOL 58"
       1959       "FACT"
       1959       "COBOL"
       1959       "RPG"
       1962       "APL"
```

В результате при вызове словаря можно, выбрав любой год, узнать, какие языки программирования были созданы в этом году:

```
[15]: # Инициализация словаря:
      dict2 = Dict()
```

```
[15]: Dict{Any, Any}()
```

```
[17]: dict = Dict{Integer,Vector{String}}()
```

```
[17]: Dict{Integer, Vector{String}}()
```

3.3 DataFrames

Работа с данными, записанными в структуре DataFrame, позволяет использовать индексацию и получить доступ к столбцам по заданному имени заголовка или по индексу столбца. На примере с данными о языках программирования и годах их создания зададим структуру DataFrame:

```
[19]: # Пример определения в словаре языков программирования, созданных в 2003 году:
      dict[2003]
```

```
[19]: 2-element Vector{String}:
       "Groovy"
       "Scala"
```

Если требуется получить доступ к столбцам по имени заголовка, то необходимо добавить к имени заголовка двоеточие:

```
[20]: # Подгружаем пакет DataFrames:
      using DataFrames
      # Задаём переменную со структурой DataFrame:
      df = DataFrame(year = P[:,1], language = P[:,2])
      # Вывод всех значения столбца year:
      df[!,:year]
      # Получение статистических сведений о фрейме:
      describe(df)
```

[20]: 2×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	year	1982.99	1951	1986.0	2014	0	Int64
2	language		ALGOL 58		dBase III	0	String31

В Julia это означает, что имена заголовков обрабатываются как символы. Также следует иметь в виду, что вызов df[1] эквивалентен вызову df [:year]. Пакет DataFrames предоставляет возможность с помощью description получить основные статистические сведения о каждом столбце во фрейме данных:

```
[21]: # Подгружаем пакет RDatasets:
using RDatasets
# Задаём структуру данных в виде набора данных:
iris = dataset("datasets", "iris")
```

[21]: 150×5 DataFrame

Row	SepalLength	SepalWidth	PetalLength	PetalWidth	Species
	Float64	Float64	Float64	Float64	Cat...
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5.0	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa

3.4 RDatasets

С данными можно работать также как с наборами данных через пакет RDatasets языка R:

```
[22]: # Определения типа переменной:
typeof(iris)
```

```
[22]: DataFrame
```

В данном случает набор данных содержит сведения о цветах. При этом следует иметь в виду, что данные, загруженные с помощью набора данных, хранятся в виде DataFrame:

Пакет RDatasets также предоставляет возможность с помощью description получить основные статистические сведения о каждом столбце в наборе данных:

```
[23]: describe(iris)
```

[23]: 5×7 DataFrame

Row	variable	mean	min	median	max	nmissing	eltype
	Symbol	Union...	Any	Union...	Any	Int64	DataType
1	SepalLength	5.84333	4.3	5.8	7.9	0	Float64
2	SepalWidth	3.05733	2.0	3.0	4.4	0	Float64
3	PetalLength	3.758	1.0	4.35	6.9	0	Float64
4	PetalWidth	1.19933	0.1	1.3	2.5	0	Float64
5	Species		setosa		virginica	0	CategoricalValue{String, UInt8}

3.5 Работа с переменными отсутствующего типа (Missing Values)

Пакет DataFrames позволяет использовать так называемый «отсутствующий» тип:

```
[24]: # Отсутствующий тип:
      a = missing
      typeof(a)
```

```
[24]: Missing
```

Предположим есть перечень продуктов, для которых заданы калории:

```
[25]: # Определение перечня продуктов:
      foods = ["apple", "cucumber", "tomato", "banana"]
      # Определение калорий:
      calories = [missing,47,22,105]
      # Определение типа переменной:
      typeof(calories)
```

```
[25]: Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

В массиве значений калорий есть значение с отсутствующим типом:

```
[26]: # Подключаем пакет Statistics:
using Statistics
# Определение среднего значения:
mean(calories)
```

```
[26]: missing
```

При попытке получить среднее значение калорий, ничего не получится из-за наличия переменной с отсутствующим типом:

```
[27]: # Определение среднего значения без значений с отсутствующим типом:
mean(skipmissing(calories))
```

```
[27]: 58.0
```

Для решения этой проблемы необходимо игнорировать отсутствующий тип:

```
[29]: # Задание сведений о ценах:
prices = [0.85,1.6,0.8,0.6]
# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods,calories=calories)
# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods,price=prices)
# Объединение данных о калориях и ценах:
DF = innerjoin(dataframe_calories,dataframe_prices,on=:item)
```

[29]: 4×3 DataFrame

Row	item	calories	price
	String	Int64?	Float64
1	apple	*missing*	0.85
2	cucumber	47	1.6
3	tomato	22	0.8
4	banana	105	0.6

Далее показано, как можно сформировать таблицы данных и объединить их в один фрейм:

```
[30]: using FileIO
```

```
[33]: import Pkg
      Pkg.add("ImageIO")
      # Загрузка изображения:
      X1 = load("julialogo.png")
```

```
   Resolving package versions...
     Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
    Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
```

```
[33]: 512×512 Matrix{ColorTypes.RGBA{FixedPointNumbers.N0f8}}:
 RGBA(0.0, 0.0, 0.0, 0.0)  …  RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)  …  RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)  …  RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 ⋮                         ⋱
 RGBA(0.0, 0.0, 0.0, 0.0)  …  RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)  …  RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)  …  RGBA(0.0, 0.0, 0.0, 0.0)
 RGBA(0.0, 0.0, 0.0, 0.0)     RGBA(0.0, 0.0, 0.0, 0.0)
```

3.6 FileIO

В Julia можно работать с так называемыми «сырыми» данными, используя пакет FileIO

Попробуем посмотреть, как Julia работает с изображениями. Подключим соответствующий пакет:

```
[34]: @show typeof(X1);
```

```
typeof(X1) = Matrix{ColorTypes.RGBA{FixedPointNumbers.N0f8}}
```

```
[35]: @show size(X1);
```

```
size(X1) = (512, 512)
```

Загрузим изображение (в данном случае логотип Julia):

Julia хранит изображение в виде множества цветов:

[36]:
```
# Загрузка пакетов:
import Pkg
Pkg.add("DataFrames")
Pkg.add("Statistics")
using DataFrames
using CSV
import Pkg
Pkg.add("Plots")
```

```
  Resolving package versions...
    Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
   Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
  Resolving package versions...
    Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
   Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
  Resolving package versions...
    Project No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
   Manifest No packages added to or removed from `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
```

3.7 Обработка данных: стандартные алгоритмы машинного обучения в Julia

Кластеризация данных. Метод k-средних. Задача кластеризации данных заключается в формировании однородной группы упорядоченных по какому-то признаку данных. Метод k-средних позволяет минимизировать суммарное квадратичное отклонение точек кластеров от центров этих кластеров.

Рассмотрим задачу кластеризации данных на примере данных о недвижимости. Файл с данными houses.csv содержит список транзакций с недвижимостью в районе Сакраменто, о которых было сообщено в течение определённого числа дней. Сначала подключим необходимые для работы пакеты:

[37]:
```
# Загрузка данных:
houses = CSV.File("houses.csv") |> DataFrame
```

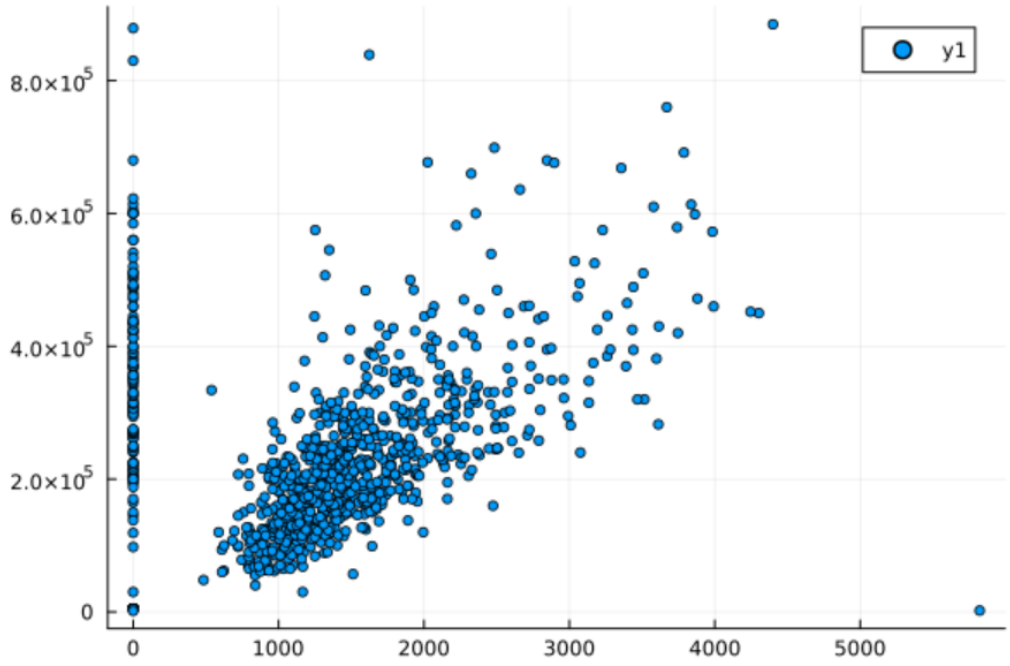
[37]: 985×12 DataFrame

Row	street	city	zip	state	beds	baths	sq_ft	type	sale_date	price	latitude	longitude
	String	String15	Int64	String3	Int64	Int64	Int64	String15	String31	Int64	Float64	Float64
1	3526 HIGH ST	SACRAMENTO	95838	CA	2	1	836	Residential	Wed May 21 00:00:00 EDT 2008	59222	38.6319	-121.435
2	51 OMAHA CT	SACRAMENTO	95823	CA	3	1	1167	Residential	Wed May 21 00:00:00 EDT 2008	68212	38.4789	-121.431
3	2796 BRANCH ST	SACRAMENTO	95815	CA	2	1	796	Residential	Wed May 21 00:00:00 EDT 2008	68880	38.6183	-121.444
4	2805 JANETTE WAY	SACRAMENTO	95815	CA	2	1	852	Residential	Wed May 21 00:00:00 EDT 2008	69307	38.6168	-121.439
5	6001 MCMAHON DR	SACRAMENTO	95824	CA	2	1	797	Residential	Wed May 21 00:00:00 EDT 2008	81900	38.5195	-121.436

Затем загрузим данные:

```
[38]: # Построение графика:
using Plots
plot(size=(500,500),leg=false)
x = houses[!,:sq__ft]
y = houses[!,:price]
scatter(x,y,markersize=3)
```

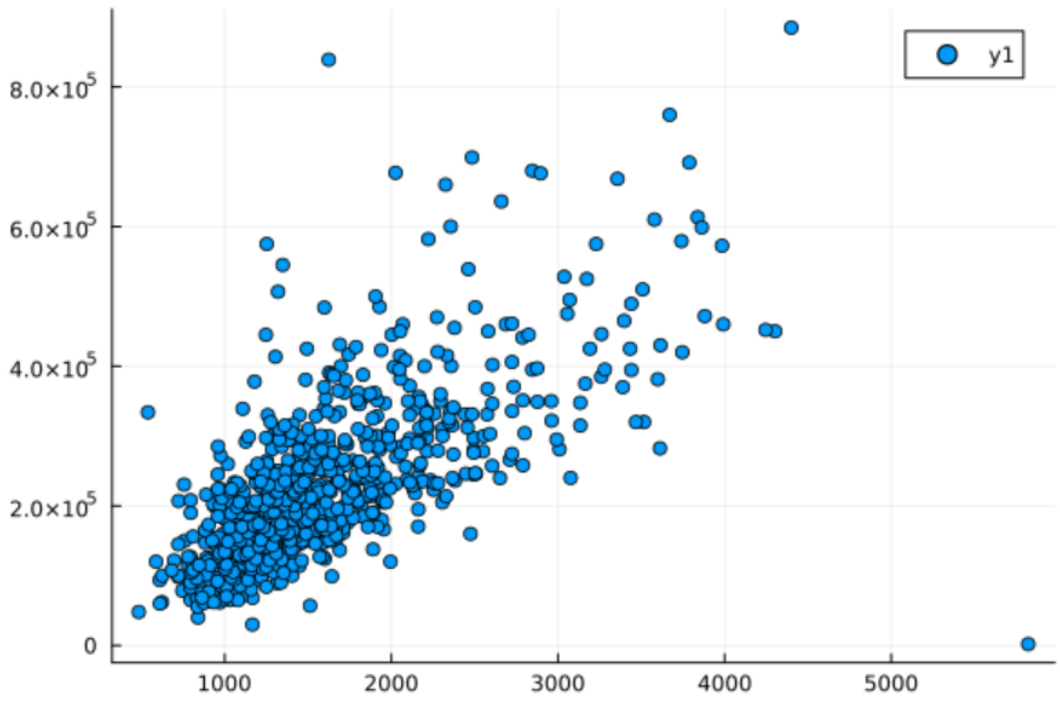
[38]:



Построим график цен на недвижимость в зависимости от площади:

```
[39]: # Фильтрация данных по заданному условию:
filter_houses = houses[houses[!,:sq__ft].>0,:]
# Построение графика:
x = filter_houses[!,:sq__ft]
y = filter_houses[!,:price]
scatter(x,y)
```

[39]:



Как видно из графика , имеются так называемые «артефакты», т.е. проявляются отсутствующие или невозможные сведения в исходных данных, например, цены на недвижимость нулевой площади. Для того чтобы избавиться от такого эффекта, можно отфильтровать и исключить такие значения, получить более корректный график цен:

```
[7]: # Подключение пакета Statistics:
using Statistics
# Определение средней цены для определённого типа домов:
by(filter_houses,:type,filter_houses->mean(filter_houses[!,:price]))
```

Используя для фильтрации значений функцию by пакета DataFrames и для вычисления среднего значения функцию mean пакета Statistics, можно посмотреть среднюю цену домов определённого типа:

```
[22]: import Pkg
Pkg.add("Clustering")
using Clustering
# Добавление данных :latitude и :longitude в новый фрейм:
X = filter_houses[[:latitude,:longitude]]
# Конвертация данных в матричный вид:
X = convert(Matrix{Float64}, X)
```

```
Resolving package versions...
Installed Distances ──────── v0.10.12
Installed NearestNeighbors ─ v0.4.27
```

Отфильтровав таким образом данные, можно приступить к формированию кластеров. Сначала подключаем необходимые пакеты и формируем данные в нужном виде:

```
[28]: k = length(unique(filter_houses[!,:zip]))
```

```
[28]: 66
```

4 Выводы

В результате выполнения лабораторной работы, я изучил специализированные пакеты Julia для обработки данных.